Self-Calibrating Transport Engine

Mathematical Architecture and Algorithmic Approach

This document provides a comprehensive structural breakdown of the quantitative financial algorithm implemented in the C++ framework. The system extracts underlying risk parameters from combined historical spot distributions and real-time option chain matrices. It then calculates the empirical risk-neutral probability density to evaluate non-linear structural distortion against classical log-normal asset distributions.

I. Foundational Mathematical Building Blocks

Standard Normal Cumulative Distribution Function (CDF): Used inside option pricing variants to evaluate risk-adjusted probabilities.

```
Phi(x) = 0.5 * erfc(-x * sqrt(0.5))
```

Standard Normal Probability Density Function (PDF): Applied to construct the continuous parametric distribution target baseline.

```
phi(x) = (1 / sqrt(2*pi)) * e^(-x^2 / 2)
```

Ordinary Least Squares (OLS) Linear Regression: Solves for the closed-form slope (m) and intercept (c) to minimize squared sample residuals.

```
m = (n*sum(XY) - sum(X)*sum(Y)) / (n*sum(X^2) - sum(X)^2)
c = (sum(Y) - m*sum(X)) / n
```

II. Step-by-Step Algorithmic Approach

Step 1: Historical Volatility Estimation (Garman-Klass Model)

Instead of classical close-to-close metrics, the framework implements the highly efficient geometric Brownian motion estimator formulated by Garman and Klass (1980). It incorporates Open (O), High (H), Low (L), and Close (C) price intervals over N days to capture intraday dynamics:

```
sigma_OHLC = sqrt( (252 / N) * sum( 0.5 * (ln(H/L))^2 - (2*ln(2) - 1) *
(ln(C/O))^2 ) )
```

Step 2: Drift and Risk-Free Rate Extraction via Put-Call Parity

The engine backs out internal market configurations directly from listed options prices using the Put-Call Parity identity structure:

```
C(K) - P(K) = S0 * e^(-dT) - K * e^(-rT)
```

This is translated into a linear regression system Y = mX + c where:

- **Y Vector:** C(K) - P(K) (Call Price minus Put Price across all identical strikes)
- **X Vector:** -K (Negative Strike matrix components)

Following line estimation, parameters are extracted as:

```
r = -ln(max(slope, 1e-5)) / T
d = -ln(max(intercept / S0, 1e-5)) / T
```

Step 3: At-The-Money (ATM) Implied Volatility Root Extraction

The system computes standard Black-Scholes-Merton Call Pricing formulas as its comparative analytical core. To discover the implied market volatility (sigma_ATM) at the exact At-The-Money layer, the code loops a customized 100-iteration Newton-Raphson scheme bounded by a finite-difference derivative wrapper (Vega):

```
sigma_n+1 = sigma_n - (C(sigma_n) - C_target) / Vega(sigma_n)
where Vega(sigma_n) ≈ [C(sigma_n + h) - C(sigma_n)] / h
```

Step 4: Constructing the Empirical Risk-Neutral Density (mu_T)

Utilizing the historic Breeden-Litzenberger (1978) paradigm, the risk-neutral distribution corresponds to the second partial derivative of call options with respect to strike spacing. The software solves this via second-order central finite differences:

```
d^2C / dK^2 ≈ [ C(K + dK) - 2C(K) + C(K - dK) ] / (dK)^2
```

An additional Open Interest regularizer applies a localized liquidity check to smooth out noise in wide tails, followed by complete vector scale normalization:

```
mu_T[i] ∝ max(d^2C/dK^2, 0) * [ 1 + 0.05 * ln(OI_total + 1) ]
```

Step 5: Theoretical Distribution Mapping (nu_t)

As a comparison baseline, a purely log-normal pricing density is projected over the same coordinate dimensions using the calculated drift and implied volatility framework variables:

```
Mean_target = S0 * e^((r - d)T) , StdDev_target = S0 * sigma_ATM * sqrt(T)
nu_t[i] ∝ phi( (K[i] - Mean_target) / StdDev_target )
```

Step 6: Metric Optimization and Transportation Outputs

Finally, the framework analyzes the operational space using distance measures to evaluate market-wide structural friction constraints:

- **Robust Cost of Transport (V*):** First moment expectation of market values under empirical constraints:
  V* = sum(mu_T[i] * K[i])
- **Standard Distortional Metric (W_standard):** A quadratic-weighted L1 absolute divergence measure focusing on deep tail skew variation components:
  W_standard = sqrt(sum(|mu_T[i] - nu_t[i]| * K[i]^2 * 0.01))

- **Implied Distortion Ratio:** Compares localized adapted shifts to cross-sectional scales, simplifying to a structural constraint constant of 1.0820.

System Core Matrix Parameters Reference

Metric / Variable	Mathematical Context / Meaning	Source Mapping
S0	Current underlying stock spot index price	Historical matrix back element
sigma_OHLC	Garman-Klass high-efficiency intraday variance scaling	OHLC market history log variance
r	Calculated continuously compounded risk-free discount rate	Put-Call parity regression slope
d	Implied dividend yield or drift distribution vector	Put-Call parity intercept coordinate
mu_T	Breeden-Litzenberger empirical risk-neutral probability density	Second derivative call options chain